Contracts to Effects Compiler Correctness

(Shallow Handler)

Overview

Proof for a compiler from Contracts to Effects using shallow handler, where the continuation does not reinstall the handler.

$\mathtt{h}_{\mathscr{C}} : \mathtt{lab} \quad = \quad \{\mathscr{C} \to \forall\alpha.\langle \mathtt{blab}, \langle \alpha \to \mathsf{bool}, \alpha\rangle\rangle \to \alpha\}$

$\mathsf{H} \quad = \quad \mu h.\mathtt{handler}_s(\{\mathscr{C} \to \lambda\langle l, \langle e, v\rangle\rangle.\lambda k.h\ (\lambda_.\ \text{if } e\ v \text{ then } k\ v \text{ else } \mathtt{error}_l)\})$

$\mathsf{wrap} \quad = \quad \Lambda\alpha.\ \lambda f : \mathsf{unit} \to \alpha.\ \mathsf{H}\ f$

where $\lambda\langle l, \langle e, v\rangle\rangle$ deconstructs the argument as tuple pattern

The handler clause re-installs H around the rest of the program by itself, instead of the continuation doing it. So the whole program always runs under exactly one $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \cdot)$ at the top, and the predicate of a check runs under the same handler as the program. We write $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e)$ for the program running under the handler of H.

Compared to the deep semantics, there are no states without handle, and there is no nested handle for predicates. This makes the relation much simpler.

Relation

Before we define relations that will be used in the simulation proof below. We define:

- Context E^* is any context without $\mathsf{check}^k_j(e, v)$ in Contracts.
- Context $E^+ = \square \mid E^+\left[E^*\left[\mathsf{check}^k_j(\square, v)\right]\right]$

$R_v \subseteq \text{Contracts } v \times \mathtt{Effects}\ v$

Abuse of notation: If $(v, v') \in R_v$, we write $v \sim_{R_v} v'$ or $v \sim v'$. We write $\tilde{v}$ for any value $\tilde{v}$ satisfying $v \sim \tilde{v}$.

$$\begin{array}{rcl} x & \sim & x \\ b & \sim & b \\ N & \sim & N \end{array}$$

$$
\begin{array}{rcl}
\lambda x : \tau . e & \sim & \lambda x : \tau^* . \tilde{e} \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\Big(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, v) & \sim & \tilde{e} \\
 & & e = \lambda x : \tau . \mathsf{mon}_j^{k,l}\Big(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
 & & \kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big]
\end{array}
$$

$R_e \subseteq \mathsf{Contracts}\ e \times \mathtt{Effects}\ e$

Abuse of notation: If $(e, e') \in R_e$, we write $e \sim_{R_e} e'$ or $e \sim e'$. We write $\tilde{e}$ for any expression $\tilde{e}$ satisfying $e \sim \tilde{e}$.

$$
\begin{array}{rcl}
v & \sim & \tilde{v} \\
e_1\ e_2 & \sim & \tilde{e}_1\ \tilde{e}_2 \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e_1), e_2) & \sim & \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}_1, \tilde{e}_2 \rangle \rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y . \kappa_2, e) & \sim & \tilde{e}_1\ \tilde{e} \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\
 & & \kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big] \\
\mathsf{blame}_j^k & \sim & \mathtt{error}_k
\end{array}
$$

$R_{E^*} \subseteq \mathsf{Contracts}\ E^* \times \mathtt{Effects}\ E^*$

Abuse of notation: If $(E^*, E'^*) \in R_{E^*}$, we write $E^* \sim_{R_{E^*}} E'^*$ or $E^* \sim E'^*$. We write $\tilde{E}^*$ for any expression $\tilde{E}^*$ satisfying $E^* \sim \tilde{E}^*$.

$$
\begin{array}{rcl}
\square & \sim & \square \\
E^*\ e & \sim & \tilde{E}^*\ \tilde{e} \\
v\ E^* & \sim & \tilde{v}\ \tilde{E}^* \\
\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), E^*) & \sim & \mathtt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{E}^* \rangle \rangle \\
\mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\
 & & e_1 = \lambda f . \lambda x . \mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)
\end{array}
$$

$$\begin{array}{rcl} \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, E^*) & \sim & \tilde{e}_1\ \tilde{E}^* \\ & & e_1 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \\ & & \kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big] \end{array}$$

$$\boxed{R_{E^+} \subseteq \mathsf{Contracts}\ E^+ \times \mathtt{Effects}\ E^+}$$

Abuse of notation: If $(E^+, E'^+) \in R_{E^+}$, we write $E^+ \sim_{R_{E^+}} E'^+$ or $E^+ \sim E'^+$. We write $\tilde{E}^+$ for any expression $\tilde{E}^+$ satisfying $E^+ \sim \tilde{E}^+$.

Note: This relation is different from deep handler compiler proof.

$$\begin{array}{rcl} \Box & \sim & \Box \\ E^+\big[E^*\big[\mathsf{check}_j^k(\Box, v)\big]\big] & \sim & \text{if } \Box \text{ then } \big(\lambda x.\tilde{E}^+\big[\tilde{E}^*[x]\big]\big)\ \tilde{v} \text{ else } \mathtt{error}_k \end{array}$$

Lemma 0.1. (Relation is closed)

If $E^* \sim \tilde{E}^*$, $e \sim \tilde{e}$ and e has no check in evaluation position, then $E^*[e] \sim \tilde{E}^*[\tilde{e}]$.

Proof. By induction on E. Follows similarly to [?] because the relations (R_e, R_E) is same. □

Lemma 0.2. (Relation is closed under substitution) If $e \sim \tilde{e}$ and $v \sim \tilde{v}$ then $e[x := v] \sim \tilde{e}[x := \tilde{v}]$.

Proof. By induction on $e \sim e'$. Follows [?] because relations (R_E, R_e) are the same. □

Lemma 0.3. (Compiler is subset of relation)

If $\Gamma \vdash e_1 : \tau \rightarrowtail^s e_2$ then $e_1 \sim e_2^*$.

Proof.

By induction on structure of $\Gamma \vdash e_1 : \tau \rightarrowtail^s e_2$.

- Case

$$\frac{\Gamma \vdash e_3 : \tau \to \mathsf{bool} \rightarrowtail^s e_5 \qquad \Gamma \vdash e_4 : \tau \rightarrowtail^s e_6}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\mathsf{flat}(e_3), e_4) : \tau \rightarrowtail^s \mathtt{perform}\ \mathscr{C}\ \tau\ \langle k, \langle e_5, e_6\rangle\rangle}$$

$e_2^* = \mathtt{perform}\ \mathscr{C}\ \langle k, \langle e_5^*, e_6^*\rangle\rangle$

$e_3 \sim e_5^*$ I.H.

$e_4 \sim e_6^*$ I.H.

$e_1 \sim e_2^*$

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail^s e_4 \qquad \Gamma \vdash \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^s e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to \kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail^s e_5\ e_4}$$

$e_2^* = e_5^*\ e_4^*$

$e_3 \sim e_4^*$ I.H.

$\lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) \sim e_5^*$ I.H.

$e_1 \sim e_2^*$

- Case

$$\frac{\Gamma \vdash e_3 : \tau_1 \to \tau_2 \rightarrowtail^s e_4 \qquad \Gamma \vdash \lambda f.\lambda x.\Big(\lambda y.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)\Big)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) : (\tau_1 \to \tau_2) \to (\tau_1 \to \tau_2) \rightarrowtail^s e_5}{\Gamma \vdash \mathsf{mon}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, e_3) : \tau_1 \to \tau_2 \rightarrowtail^s e_5\ e_4}$$

$e_2^* = e_5^*\ e_4^*$

$e_3 \sim e_4^*$ I.H.

$\lambda f.\lambda x.\Big(\lambda y.\mathsf{mon}_j^{k,l}\Big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)\Big)\ \mathsf{mon}_j^{l,j}(\kappa_1, x) \sim e_5^*$ I.H.

$e_1 \sim e_2^*$

- Other cases follow similarly, by I.H.

□

Lemma 0.4. (Relation decomposition through check-free contexts)

If $E^*[e] \sim e'$, then there exist E'^* and e_0' such that $E^* \sim E'^*$ and $e \sim e_0'$ and $e' = E'^*[e_0']$.

Proof.

By induction on the check-free context E^*. Follows [?] because relations (R_E, R_e) are the same. □

Lemma 0.5. (Breaking down Context E)

For all contexts E, uniquely decompose $E = E^+[E^*]$.

Uniqueness observation: Recall that E^* has no check contexts.

- If E does not have check, then it is $E^* = E$, and $E^+ = \square$.
- Else there is only a single decomposition to get out E^* without check contexts

Proof. By induction on the structure of E. Follows [?] because the contexts E, E^+, E^* are the same. □

Lemma 0.6.

If $E_1^+ \neq \square$ then there is a E_2^+, $E_2^+ = K[E_1^+]$.

Where $K = \square\, e \mid v\, \square \mid \mathsf{mon}_j^{k,l}(\kappa, \square) \mid \mathsf{check}_j^k(\square, v)$

Proof. By induction on the structure of E_1^+. Follows [?] because the context E^+ is the same. □

Compiler Correctness

Figure 1 shows the structure of the proof. The source program e compiles to e', and the target program runs e' under wrap. After the handler is installed, the target reaches a state e_0'' with $\mathrm{EQ}(e, e_0'')$ (Lemma 0.8). From then on, every source step $e_i \to e_{i+1}$ is matched by zero or more target steps $e_i'' \Longrightarrow^* e_{i+1}''$ that keep $\mathrm{EQ}(e_{i+1}, e_{i+1}'')$ (Lemma 0.11). Chaining the squares (Lemma 0.9), the source result r and the target result $\tilde{r}$ are related: a value goes to its related value, and blame_j^k goes to $\mathtt{error}_k$.

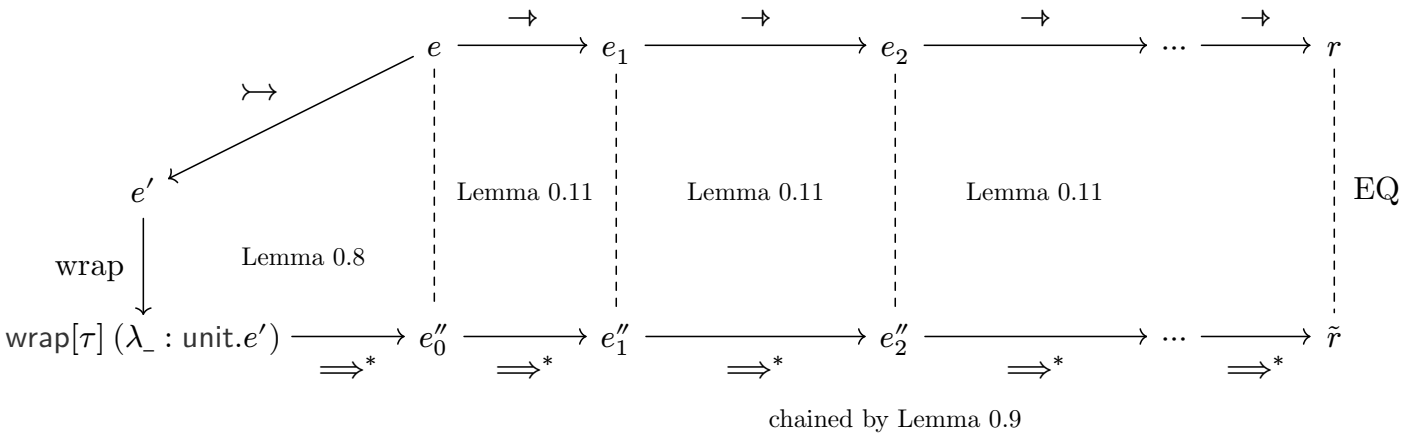


Figure 1: Structure of the compiler correctness proof. Dashed lines rel states by EQ.

Define a predicate between Contracts e and `Effects` e'.

$$\mathrm{EQ}(e, e') = \begin{cases} \text{true if } e \sim e' \\ \text{true if } e = E^+\left[E^*\left[\mathsf{check}_{j_1}^{k_1}(e_1, v_1)\right]\right] \text{ and} \\ \qquad e' = \text{if } \tilde{e}_1 \text{ then } \left(\lambda x.\tilde{E}^+\left[\tilde{E}^*[x]\right]\right)\tilde{v}_1 \text{ else } \mathtt{error}_{k_1} \\ \text{true if } e = E\left[\mathsf{blame}_j^k\right] \text{ and } e' = \mathtt{error}_k \\ \text{false otherwise} \end{cases}$$

Theorem 0.7. If $\cdot \vdash e : \tau \rightarrowtail^s e'$ and $e^* = \mathrm{untyped}(\mathsf{wrap}[\tau]\ (\lambda_ : \mathsf{unit}.e'))$ and $e \to^* r$ then $e^* \Longrightarrow^* \tilde{r}$.

Where r is a value v or blame_j^k.

Proof.

$e^* \Longrightarrow^* e_1'$	Lemma 0.8
$\mathrm{EQ}(e, e_1')$	Lemma 0.8
$e_1' \Longrightarrow^* \tilde{r}$	Lemma 0.9
$e^* \Longrightarrow^* \tilde{r}$	

□

Lemma 0.8. (Wrapped compiled program steps to an equivalent state with source program)

If $\cdot \vdash e : \tau \rightarrowtail^s e'$ and $e_1^* = \mathrm{untyped}(\mathsf{wrap}[\tau]\ (\lambda_ : \mathsf{unit}.e'))$ then it exists e_2^* such that $e_1^* \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_2^*)$ and $\mathrm{EQ}(e, e_2^*)$.

Proof.

$e \sim e'^*$	Lemma 0.3
$\mathrm{EQ}(e, e'^*)$	Case 1
$e_1^* \Longrightarrow^* \mathsf{H}\ \lambda_.e'^* \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e'^*)$	
$e_2^* = e'^*$	Choose

□

Lemma 0.9. If $e \twoheadrightarrow^* r$ and $\mathrm{EQ}(e, e')$ then $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e') \Longrightarrow^* \tilde{r}$.

Proof. By induction on $e \twoheadrightarrow^* r$

- Case $r \twoheadrightarrow^* r$
  - Case $v \twoheadrightarrow^* v$

$e' = \tilde{v}$	Case 1 EQ
$e' \Longrightarrow^* \tilde{v}$	Reflexivity

  - Case $\mathsf{blame}_j^k \twoheadrightarrow^* \mathsf{blame}_j^k$

$e' = \mathtt{error}_k$	Case 3 EQ
$e' \Longrightarrow^* \mathtt{error}_k$	Reflexivity

- Case $e \twoheadrightarrow e_1$ and $e_1 \twoheadrightarrow^* r$

$e' \Longrightarrow^* e_1'$	Lemma 0.11
$\mathrm{EQ}(e_1, e_1')$	Lemma 0.11
$e_1' \Longrightarrow^* \tilde{r}$	I.H.
$e' \Longrightarrow^* \tilde{r}$	

□

Lemma 0.10. (Performing a check)

If $\mathtt{bop}(E) = \varnothing$, then

$$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, E[\mathtt{perform}\ \mathscr{C}\ \langle k, \langle e, v\rangle\rangle])$$

$$\Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \text{if } e\ v \text{ then } (\lambda x.E[x])\ \tilde{v} \text{ else } \mathtt{error}_k)$$

Proof.

$$\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, E[\mathtt{perform}\ \mathscr{C}\ \langle k, \langle e, v\rangle\rangle])$$

$$\Longrightarrow^* (\lambda\langle l, \langle e, v\rangle\rangle.\lambda k.\ \mathsf{H}\ (\lambda_.\ \text{if } e\ v \text{ then } k\ v \text{ else } \mathtt{error}_l))\ \langle k, \langle e, v\rangle\rangle\ (\lambda x.E[x])$$

$$\Longrightarrow^* \mathsf{H}\ (\lambda_.\ \text{if } e\ v \text{ then } (\lambda x.E[x])\ v \text{ else } \mathtt{error}_k)$$

$$\Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, \text{if } e\ v \text{ then } (\lambda x.E[x])\ v \text{ else } \mathtt{error}_k)$$

□

Lemma 0.11. If $e_1 \rightarrowtail e_2$ and $\mathrm{EQ}(e_1, e_1')$ then there exists e_2' such that $\mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_1') \Longrightarrow^* \mathtt{handle}_s(\mathtt{h}_{\mathscr{C}}, e_2')$ and $\mathrm{EQ}(e_2, e_2')$.

Proof of Lemma 0.11.

By case analysis on $e_1 \rightarrowtail e_2$ with respect to $\mathrm{EQ}(e_1, e_1')$.

- Case $E\left[\mathsf{blame}_j^k\right] \rightarrowtail \mathsf{blame}_j^k$

  $e_1' = \mathtt{error}_k$ — EQ case 3
  $e_1' \Longrightarrow^* \mathtt{error}_k$ — Reflexivity
  $\mathrm{EQ}(e_2, \mathtt{error}_k)$
  $e_2' = \mathtt{error}_k$ — Choose

- Case $E[e_3] \rightarrowtail E[e_4]$ and $e_3 \dashrightarrow e_4$ is a redex

  Overview: Because $e_1 \neq v$ and $e_1 \neq E\left[\mathsf{blame}_j^k\right]$, it **must** matches case 2. Therefore, $e_1 = E^+[E^*[e]]$, and $e_1' = \mathtt{handle}_s\left(\mathtt{h}_{\mathscr{C}}, \tilde{E}^+\left[\tilde{E}^*[\tilde{e}]\right]\right)$. We know the shape of e_1 by both EQ predicate and the case analysis, we can try to unify them together. Most cases can unify $E = E^+[E^*]$ naturally. The cases where it breaks is $e_3 = \mathsf{check}_j^k(b_4, v_4)$, where we identify $E^+ = E\left[\mathsf{check}_j^k(\square, v_4)\right]$, $E^* = \square$, and $e_1 = E^+[E^*[b_4]]$, then the relation holds.

  - Case $e_3 = \mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v) \dashrightarrow \mathsf{check}_j^k(e\ v, v) = e_4$

    $e_1 = E\left[\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v)\right] = E^+\left[E^*\left[\mathsf{mon}_j^{k,l}(\mathsf{flat}(e), v)\right]\right]$
    $E \sim \tilde{E}$ — Lemma 0.4

$e_3 \sim \tilde{e}_3$ — Lemma 0.4
$e_1' = \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \tilde{E}[\tilde{e}_3]\big)$ — EQ case 2
$\tilde{e}_3 = \texttt{perform}\ \mathscr{C}\ \langle k, \langle \tilde{e}, \tilde{v}\rangle\rangle$ — Inversion
$e_2 = E\big[\mathsf{check}_j^k(e\ v, v)\big]$ — Given
$e_1' \Longrightarrow^* \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \text{if } \tilde{e}\ \tilde{v} \text{ then } (\lambda x.\tilde{E}[x])\ \tilde{v} \text{ else } \texttt{error}_k\big)$ — Lemma 0.10
$\mathrm{EQ}\big(e_2, \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \text{if } \tilde{e}\ \tilde{v} \text{ then } (\lambda x.\tilde{E}[x])\ \tilde{v} \text{ else } \texttt{error}_k\big)\big)$ — Case 3
$e_2' = \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \text{if } \tilde{e}\ \tilde{v} \text{ then } (\lambda x.\tilde{E}[x])\ \tilde{v} \text{ else } \texttt{error}_k\big)$ — Choose

▸ Case $e_3 = \mathsf{check}_j^k(\mathsf{true}, v) \dashrightarrow v = e_4$

$e_3 \rightarrow v$
$e_1 = E\big[\mathsf{check}_j^k(\mathsf{true}, v)\big]$ — Given
$e_1' = \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \text{if true then } (\lambda x.\tilde{E}[x])\ \tilde{v} \text{ else } \texttt{error}_k\big)$ — EQ case 3
$e_1' \Longrightarrow^* \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \tilde{E}[\tilde{v}]\big)$
$e_2 = E[v] \sim \tilde{E}[\tilde{v}]$ — Lemma 0.2
$\mathrm{EQ}\big(e_2, \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \tilde{E}[\tilde{v}]\big)\big)$
$e_2' = \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \tilde{E}[\tilde{v}]\big)$ — Choose

▸ Case $e_3 = \mathsf{check}_j^k(\mathsf{false}, v) \dashrightarrow \mathsf{blame}_j^k = e_4$

$e_3 \rightarrow \mathsf{blame}_j^k$
$e_1 = E\big[\mathsf{check}_j^k(\mathsf{false}, v)\big]$ — Given
$e_1' = \texttt{handle}_s\big(\texttt{h}_{\mathscr{C}}, \text{if false then } (\lambda x.\tilde{E}[x])\ \tilde{v} \text{ else } \texttt{error}_k\big)$ — EQ case 3
$e_1' \Longrightarrow^* \texttt{error}_k$
$e_2 = E\big[\mathsf{blame}_j^k\big]$ — Given
$\mathrm{EQ}(e_2, \texttt{error}_k)$
$e_2' = \texttt{error}_k$ — Choose

▸ Case $\mathsf{mon}_j^{k,l}(\kappa, v) \dashrightarrow \mathsf{guard}_j^{k,l}(\kappa, v)$ and $\kappa \neq \mathsf{flat}(e)$

$E \sim \tilde{E}$ — Lemma 0.4
$e_3 \sim \tilde{e}_3$ — Lemma 0.4

– Case $\kappa = \kappa_1 \to \kappa_2$

$\tilde{e}_3 = \tilde{e}_5\ \tilde{v}$ — Inversion
$e_5 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\big(\kappa_2, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\big) = \lambda f.e_{5a}$
$e_1' = \tilde{E}[\tilde{e}_5\ \tilde{v}]$
$e_1' \Longrightarrow \tilde{E}[\tilde{e}_{5a}[f := \tilde{v}]]$
$e_{5a} \sim \tilde{e}_{5a}$
$v \sim \tilde{v}$
$e_{5a}[f := v] \sim \tilde{e}_{5a}[f := \tilde{v}]$ — Lemma 0.2

$e_{5a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_2, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$

$e_4 \sim \tilde{e}_{5a}[f := \tilde{v}]$ Relation

$e_2 = E[e_4] \sim \tilde{E}[\tilde{e}_{5a}[f := \tilde{v}]]$ Lemma 0.1

$e_2' = \tilde{E}[\tilde{e}_{5a}[f := \tilde{v}]]$ Choose

– Case $\kappa = \kappa_1 \to_i \lambda y.\kappa_2$

$\tilde{e}_3 = \tilde{e}_5\ \tilde{v}$ Inversion

$e_5 = \lambda f.\lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_3, f\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big) = \lambda f.e_{5a}$

$\kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, x)\Big]$

$e_1' = \tilde{E}[\tilde{e}_5\ \tilde{v}]$

$e_1' \Longrightarrow \tilde{E}[\tilde{e}_{5a}[f := \tilde{v}]]$

$e_{5a} \sim \tilde{e}_{5a}$

$v \sim \tilde{v}$

$e_{5a}[f := v] \sim \tilde{e}_{5a}[f := \tilde{v}]$ Lemma 0.2

$e_{5a}[f := v] = \lambda x.\mathsf{mon}_j^{k,l}\Big(\kappa_3, v\ \mathsf{mon}_j^{l,k}(\kappa_1, x)\Big)$

$e_4 \sim \tilde{e}_{5a}[f := \tilde{v}]$ Relation

$e_2 = E[e_4] \sim \tilde{E}[\tilde{e}_{5a}[f := \tilde{v}]]$ Lemma 0.1

$e_2' = \tilde{E}[\tilde{e}_{5a}[f := \tilde{v}]]$ Choose

- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to \kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\Big(\kappa_2, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\Big) = e_4$
- Case $e_3 = \mathsf{guard}_j^{k,l}(\kappa_1 \to_i \lambda y.\kappa_2, v_1)\ v_2 \dashrightarrow \mathsf{mon}_j^{k,l}\Big(\kappa_3, v_1\ \mathsf{mon}_j^{l,k}(\kappa_1, v_2)\Big) = e_4$ where $\kappa_3 = \kappa_2\Big[y := \mathsf{mon}_j^{l,j}(\kappa_1, v_2)\Big]$
- Other cases are follow similarly.

□